



Tài liệu phân tích giải pháp

*Hệ thống Chatbot AI hỗ trợ
đặt món và thanh toán F&B*

Người thực hiện: Lê Minh Trí

TP. HỒ CHÍ MINH, 04/2026



Mục lục

Danh sách hình ảnh	2
Danh sách bảng biểu	2
1 Tổng quan về đề tài	2
1.1 Mô tả đề tài	2
1.2 Mục tiêu đề tài	2
1.3 Phạm vi hệ thống	2
2 Phân tích yêu cầu (Requirements)	3
2.1 Yêu cầu chức năng (Functional Requirements)	3
2.1.1 Giao tiếp và Xử lý ngôn ngữ tự nhiên	3
2.1.2 Quản lý Đơn hàng & Thanh toán	3
2.2 Yêu cầu phi chức năng (Non-functional Requirements)	3
3 Use-case Diagram & Scenario	4
3.1 Use-case Diagram	4
3.2 Các kịch bản sử dụng (Use-case Scenarios)	5
3.2.1 Use-case: Xem thực đơn	5
3.2.2 Use-case: Đặt món và Cung cấp thông tin giao hàng	6
3.2.3 Use-case: Yêu cầu tính tiền và Tạo mã QR thanh toán	7
3.2.4 Use-case: Xử lý Webhook và Thông báo Bếp (Chốt đơn tự động)	8
4 Kiến trúc hệ thống và Thiết kế	9
4.1 Class Diagram	9
4.2 Mô tả phương thức và module nghiệp vụ	10
4.2.1 Nhóm lớp Điều phối (Controllers)	10
4.2.2 Nhóm lớp Nghiệp vụ (Services)	10
4.2.3 Nhóm lớp Dữ liệu (Models)	11
5 Sequence Diagram và Activity Diagram	11
5.1 Tra cứu thực đơn (Xem Menu)	11
5.2 Đặt món và Cung cấp thông tin giao hàng	12
5.3 Yêu cầu tính tiền và Tạo mã QR thanh toán	14
5.4 Xử lý Webhook và Thông báo Bếp (Chốt đơn tự động)	15
6 Deployment View (Kiến trúc triển khai)	18
7 Development View (Kiến trúc phát triển)	20
8 Kết luận và Hướng phát triển	21
8.1 Kết quả đạt được	21
8.2 Hạn chế và Hướng phát triển	21



1 Tổng quan về đề tài

1.1 Mô tả đề tài

Hệ thống **Casso F&B AI Assistant** là một nền tảng Chatbot tự động hóa quy trình tư vấn, đặt món và thanh toán cho ngành hàng F&B, được triển khai trên nền tảng Telegram. Dự án được thiết kế dưới dạng MVP (Minimum Viable Product) nhằm giải quyết bài toán quá tải đơn hàng online cho tập khách hàng bận rộn.

Hệ thống ứng dụng Sức mạnh của Mô hình Ngôn ngữ Lớn (LLM - GPT-4o-mini) kết hợp cơ chế Function Calling để hiểu ngôn ngữ tự nhiên của khách hàng. Đặc biệt, hệ thống tích hợp giải pháp thanh toán mã QR động thông qua payOS Webhook, giúp tự động hóa khâu đối soát tài chính và tự động điều phối thông tin đơn hàng đến bộ phận Bếp một cách chính xác.

1.2 Mục tiêu đề tài

- **Tự động hóa tư vấn:** Xây dựng AI có khả năng hiểu ngữ cảnh, nhớ lịch sử chọn món và giao tiếp linh hoạt tự nhiên với khách hàng.
- **Tự động hóa đối soát:** Tích hợp cổng thanh toán payOS để tạo mã QR động, lắng nghe Webhook để tự động chốt đơn khi dòng tiền về mà không cần sự can thiệp của con người.
- **Tối ưu hóa vận hành:** Phân loại luồng dữ liệu thông minh (Giao hàng tận nơi vs Dùng tại quán) và phân phối Bill tổng hợp trực tiếp đến Group Telegram của bộ phận Bếp.

1.3 Phạm vi hệ thống

1. Đối tượng sử dụng (Actors):

- **Khách hàng (User):** Nhắn tin với Bot để xem menu, đặt món, cung cấp thông tin giao hàng và quét mã thanh toán.
- **Nhà Bếp / Shipper (Kitchen Group):** Nhận hóa đơn tổng hợp và thông tin giao hàng tự động từ Bot để tiến hành làm món.

2. Các module chức năng chính:

- **Module AI Chat & Context Memory:** Quản lý phiên chat (Session), phân tích Intent, ép khuôn hành vi bằng System Prompt (Guardrails).
- **Module Menu & Database:** Quản lý thực đơn trên MongoDB, tìm kiếm linh hoạt bằng Regex.
- **Module Payment & Webhook:** Gọi API tạo Link thanh toán, lắng nghe tín hiệu Webhook từ payOS và cập nhật trạng thái đơn (Pending → Paid).

2 Phân tích yêu cầu (Requirements)

2.1 Yêu cầu chức năng (Functional Requirements)

2.1.1 Giao tiếp và Xử lý ngôn ngữ tự nhiên

- **FR-AI-01:** Bot phải có khả năng hiểu các từ viết tắt, gõ sai chính tả cơ bản của khách hàng để ánh xạ đúng vào tên món ăn trong cơ sở dữ liệu.
- **FR-AI-02:** Bot phải lưu trữ lịch sử hội thoại (Context Memory) để khách hàng có thể gọi món liên tiếp mà không cần lặp lại ngữ cảnh.
- **FR-AI-03:** Bot phải từ chối khéo léo các yêu cầu nằm ngoài phạm vi nghiệp vụ F&B (như giải toán, viết code) để tránh bị thao túng (Prompt Injection).

2.1.2 Quản lý Đơn hàng & Thanh toán

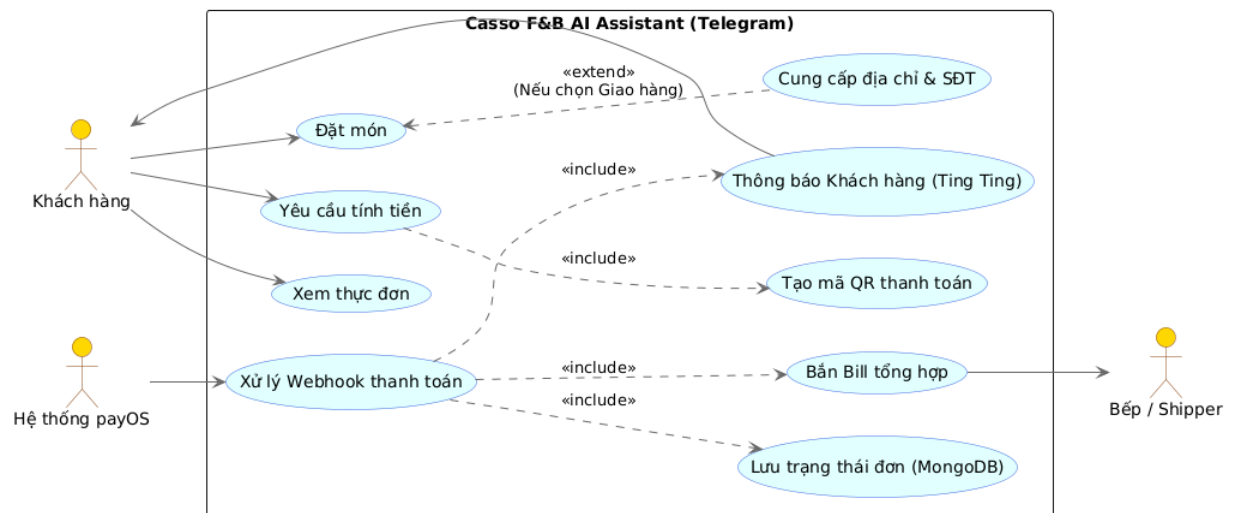
- **FR-ORD-01:** Hệ thống tự động tính toán tổng tiền dựa trên size và số lượng do AI trích xuất.
- **FR-ORD-02:** Hệ thống phải phân luồng logic: Nếu khách "Giao hàng", bắt buộc phải thu thập địa chỉ và số điện thoại trước khi tạo mã thanh toán.
- **FR-PAY-01:** Hệ thống tạo mã VietQR động chứa đúng số tiền và mã đơn hàng (Order-Code).
- **FR-WEB-01:** Hệ thống cung cấp API Endpoint (Webhook) để nhận tín hiệu thanh toán thành công từ payOS và tự động gửi tin nhắn "Ting Ting" cho khách hàng.
- **FR-WEB-02:** Sau khi nhận thanh toán, hệ thống tự động soạn Bill tổng hợp và gửi vào Group Telegram của bộ phận Bếp.

2.2 Yêu cầu phi chức năng (Non-functional Requirements)

- **Độ trễ (Latency):** Thời gian phản hồi tin nhắn văn bản thông thường của AI không vượt quá 3-5 giây.
- **Tính toàn vẹn dữ liệu (Integrity):** Thông tin đơn hàng (Pending Orders) phải được lưu trữ trong MongoDB để đảm bảo không mất đơn khi server khởi động lại.
- **Tính sẵn sàng (Availability):** Endpoint Webhook phải luôn trong trạng thái sẵn sàng nhận tín hiệu từ ngân hàng (Uptime 99.9%).

3 Use-case Diagram & Scenario

3.1 Use-case Diagram



Hình 1: Biểu đồ Use-case hệ thống Bot



3.2 Các kịch bản sử dụng (Use-case Scenarios)

3.2.1 Use-case: Xem thực đơn

Use case name	Xem thực đơn (Tra cứu Menu)
Actors	Khách hàng
Description	Khách hàng yêu cầu AI cung cấp danh sách món ăn, thức uống của quán kèm theo giá tiền.
Trigger	Khách hàng nhấn các từ khóa liên quan đến thực đơn (VD: "menu", "cho xem thực đơn", "quán có món gì").
Pre-Condition(s)	1. Khách hàng có tài khoản Telegram và đang mở khung chat với Bot. 2. Hệ thống Backend và MongoDB đang hoạt động bình thường.
Post-Condition(s)	Khách hàng nhận được danh sách thực đơn được định dạng dễ đọc.
Normal Flow	1. Khách hàng nhấn tin yêu cầu xem menu. 2. Bot (AI) phân tích ý định và tự động gọi công cụ <code>search_menu</code> với tham số từ khóa là chuỗi rỗng. 3. Backend truy vấn toàn bộ dữ liệu món ăn đang <code>available</code> từ MongoDB. 4. Hệ thống định dạng lại dữ liệu và trả kết quả về cho AI. 5. AI tổng hợp và gửi danh sách thực đơn chi tiết cho khách hàng.
Exception Flow	3a. Nếu mất kết nối Database, hàm truy vấn bị lỗi → Hệ thống bắt lỗi (try-catch) và trả về câu thông báo: "Hệ thống đang lỗi, không thể tra cứu menu lúc này". AI sẽ dùng câu này để xin lỗi khách.
Alternative Flow	1a. Khách hàng không yêu cầu xem toàn bộ menu mà chỉ hỏi một món cụ thể (VD: "Có trà sữa matcha không?") → AI trích xuất từ khóa "matcha" và gọi <code>search_menu("matcha")</code> để chỉ trả về thông tin của món đó.

Bảng 1: Mô tả Use-case Xem thực đơn



3.2.2 Use-case: Đặt món và Cung cấp thông tin giao hàng

Use case name	Đặt món và Cung cấp thông tin giao hàng
Actors	Khách hàng
Description	Khách hàng chọn món (tên, size, số lượng) và cung cấp thông tin nhận hàng để chuẩn bị cho việc chốt đơn.
Trigger	Khách hàng nhấn tin order món (VD: "Cho mình 1 ly trà đường").
Pre-Condition(s)	Người dùng đang trong phiên hội thoại với Bot.
Post-Condition(s)	Toàn bộ thông tin món ăn và địa chỉ giao hàng được lưu trữ tạm thời trong Bộ nhớ Ngữ cảnh (Context Memory) của AI.
Normal Flow	1. Khách hàng nhấn tin đặt món. 2. AI phân tích, đối chiếu với menu và trích xuất Tên món, Size, Số lượng. 3. AI xác nhận lại các món khách vừa đặt. 4. Khách hàng đồng ý. 5. AI hỏi khách hàng hình thức nhận hàng (Tại quán, Takeaway hay Giao hàng tận nơi). 6. Khách hàng chọn "Giao hàng tận nơi". 7. AI bắt buộc hỏi Tên tòa nhà/địa chỉ và Số điện thoại liên hệ. 8. Khách hàng cung cấp đủ thông tin.
Exception Flow	2a. Khách hàng đặt món không có trong thực đơn → AI sẽ khéo léo báo quán không có món này và gợi ý các món tương tự. 8a. Khách hàng chỉ cung cấp địa chỉ mà quên Số điện thoại → AI dựa vào System Prompt để phát hiện thiếu sót và tiếp tục hỏi xin số điện thoại cho đến khi đủ thông tin.
Alternative Flow	6a. Khách hàng chọn "Dùng tại quán" hoặc "Ghé lấy" → Hệ thống bỏ qua bước 7 và 8, tự động gán địa chỉ là "Tại quán" và SĐT là "Không có".

Bảng 2: Mô tả Use-case Đặt món và Cung cấp thông tin

3.2.3 Use-case: Yêu cầu tính tiền và Tạo mã QR thanh toán

Use case name	Yêu cầu tính tiền và Tạo mã QR thanh toán
Actors	Khách hàng
Description	Khách hàng chốt đơn, hệ thống tính toán tổng tiền, lưu đơn hàng vào DB và sinh mã VietQR động để khách chuyển khoản.
Trigger	Khách hàng nhấn các từ khóa chốt đơn (VD: "tính tiền", "chốt đơn", "thanh toán").
Pre-Condition(s)	Khách hàng đã chọn ít nhất 1 món và cung cấp đủ thông tin giao hàng theo quy định của AI.
Post-Condition(s)	Đơn hàng được lưu vào MongoDB với trạng thái PENDING . Ảnh QR Code được gửi ra màn hình chat.
Normal Flow	1. Khách hàng nhấn yêu cầu tính tiền. 2. AI tự động gọi công cụ <code>calculate_total</code> truyền vào danh sách món. 3. Backend tính tổng tiền dựa trên giá gốc trong DB và xuất hóa đơn chi tiết (Bill Details). 4. AI in hóa đơn ra màn hình và hỏi khách có muốn thanh toán không. 5. Khách xác nhận thanh toán. 6. AI gọi công cụ <code>generate_qr_code</code>. 7. Backend sinh mã <code>orderId</code> ngẫu nhiên, lưu toàn bộ dữ liệu đơn hàng xuống MongoDB với trạng thái PENDING. 8. Backend gọi API của payOS để lấy link thanh toán và tạo ảnh QR Code. 9. Bot gửi ảnh QR Code cho khách hàng kèm lời dặn quét mã.
Exception Flow	7a. Lỗi kết nối MongoDB → Hệ thống không thể lưu đơn, trả về thông báo lỗi cho người dùng. 8a. API payOS không phản hồi hoặc sai cấu hình API Key → Backend bắt lỗi và gửi thông báo: "Không thể tạo mã QR thanh toán lúc này."
Alternative Flow	4a. Khách hàng đổi ý, muốn thêm/bớt món sau khi xem hóa đơn → AI quay trở lại luồng Đặt món và cập nhật lại Context Memory.

Bảng 3: Mô tả Use-case Yêu cầu tính tiền và Tạo QR

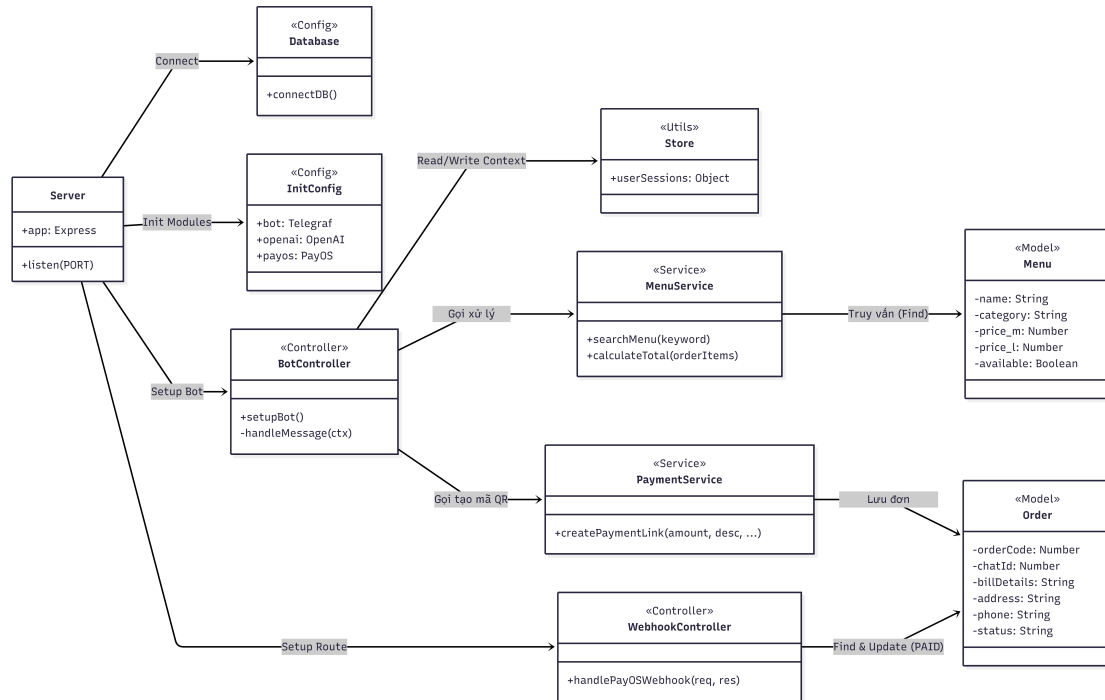
3.2.4 Use-case: Xử lý Webhook và Thông báo Bếp (Chốt đơn tự động)

Use case name	Xử lý Webhook và Thông báo Bếp
Actors	Hệ thống payOS, Bộ phận Bếp / Shipper
Description	Hệ thống tự động nhận tín hiệu tiền về từ payOS, chốt đơn cho khách và phân phối thông tin cho Bếp để làm món.
Trigger	Khách hàng chuyển khoản thành công, ngân hàng gạch nợ và payOS kích hoạt HTTP POST Request gửi về Server.
Pre-Condition(s)	Đơn hàng mang mã <code>orderId</code> đang tồn tại trong MongoDB với trạng thái <code>PENDING</code> . Link Webhook đã được cấu hình chính xác trên trang quản trị payOS.
Post-Condition(s)	Đơn hàng chuyển sang trạng thái <code>PAID</code> . Bộ phận Bếp nhận được lệnh làm món.
Normal Flow	1. API payOS gửi một payload dạng JSON chứa <code>orderId</code> về endpoint <code>/payos-webhook</code> của Server. 2. Controller tiếp nhận, kiểm tra tính hợp lệ của giao dịch (mã code = "00"). 3. Backend truy vấn MongoDB tìm đơn hàng có mã <code>orderId</code> tương ứng. 4. Backend cập nhật trạng thái đơn hàng từ <code>PENDING</code> thành <code>PAID</code> và lưu lại. 5. Backend gọi API Telegram gửi trực tiếp tin nhắn "Ting ting" thông báo thành công cho Khách hàng. 6. Backend định dạng lại hóa đơn, kèm theo địa chỉ và SĐT, gửi vào Group Telegram của bộ phận Bếp. 7. Server trả về HTTP 200 OK để payOS kết thúc phiên gọi Webhook.
Exception Flow	3a. Không tìm thấy <code>orderId</code> trong Database (có thể do lỗi đồng bộ hoặc đơn cũ đã bị xóa) → Hệ thống ghi log cảnh báo lỗi và không gửi tin nhắn. 7a. Server bị sập ngay thời điểm payOS gọi Webhook → payOS sẽ tự động thực hiện cơ chế Retry (gửi lại Webhook nhiều lần) cho đến khi Server online và trả về HTTP 200 OK.
Alternative Flow	(Không có) Luồng xử lý kỹ thuật diễn ra hoàn toàn tự động phía Backend.

Bảng 4: Mô tả Use-case Xử lý Webhook thanh toán

4 Kiến trúc hệ thống và Thiết kế

4.1 Class Diagram



Hình 2: Class Diagram - Kiến trúc hệ thống Bot



4.2 Mô tả phương thức và module nghiệp vụ

4.2.1 Nhóm lớp Điều phối (Controllers)

Nhóm này đóng vai trò là cầu nối tiếp nhận các tương tác từ nền tảng bên ngoài (Telegram, payOS) và điều hướng luồng xử lý xuống các lớp nghiệp vụ.

Tên module/lớp	Phương thức	Mô tả chức năng
BotController	setupBot()	Khởi tạo và thiết lập các sự kiện lắng nghe tin nhắn từ giao diện Telegram.
	handleMessage(ctx)	Nhận tin nhắn, quản lý Session, gọi API OpenAI để phân tích NLP và kích hoạt Function Calling.
WebhookController	handlePayOSWebhook()	Lắng nghe POST request từ payOS, kiểm tra mã đơn, cập nhật trạng thái và tự động bắn thông báo (cho khách và cho Bếp).

Bảng 5: Mô tả các phương thức nhóm lớp Điều phối

4.2.2 Nhóm lớp Nghiệp vụ (Services)

Nhóm này xử lý các logic tính toán cốt lõi của ứng dụng như tìm kiếm món ăn, tính tiền và khởi tạo mã QR thanh toán.

Tên module/lớp	Phương thức	Mô tả chức năng
MenuService	searchMenu(keyword)	Tìm kiếm món ăn trong MongoDB bằng Regex. Trả về toàn bộ menu nếu keyword rỗng.
	calculateTotal(orderItems)	Đối chiếu giá gốc trong Database để tính tổng tiền và tạo hóa đơn chi tiết (Bill Details) dạng text.
PaymentService	createPaymentLink(...)	Tạo mã đơn (OrderCode), lưu đơn hàng vào Database và gọi SDK payOS để sinh mã QR thanh toán động.

Bảng 6: Mô tả các phương thức nhóm lớp Nghiệp vụ

4.2.3 Nhóm lớp Dữ liệu (Models)

Nhóm này định nghĩa cấu trúc dữ liệu (Schema) và tương tác trực tiếp với cơ sở dữ liệu MongoDB.

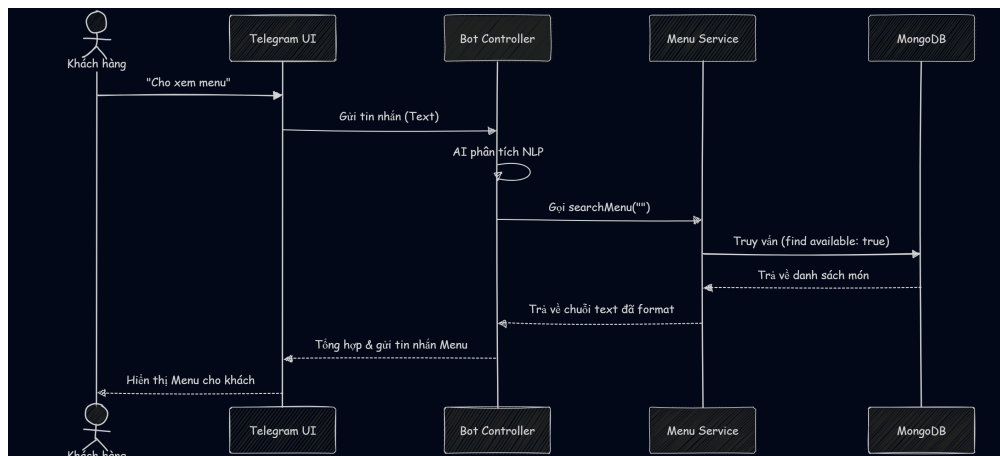
Tên module/lớp	Thuộc tính chính	Mô tả chức năng
Menu	name, category	Tên và danh mục của món ăn.
	price_m, price_l	Giá tiền tương ứng với từng kích cỡ (Size M, Size L).
	available	Trạng thái cho phép đặt hàng (Còn hàng hay Hết hàng).
Order	orderCode, chatId	Mã đơn hàng độc nhất và ID định danh của khách hàng trên Telegram.
	billDetails, address	Nội dung hóa đơn tổng hợp và thông tin giao hàng (tòa nhà, số điện thoại).
	status	Trạng thái thanh toán của đơn hàng (PENDING hoặc PAID).

Bảng 7: Mô tả cấu trúc dữ liệu nhóm lớp Models

5 Sequence Diagram và Activity Diagram

5.1 Tra cứu thực đơn (Xem Menu)

- Sequence Diagram:



Hình 3: Sequence Diagram - Tra cứu thực đơn

- **Mục đích:** Thể hiện quy trình khách hàng yêu cầu xem menu và hệ thống Bot phối hợp với Cơ sở dữ liệu để trả về danh sách các món ăn kèm giá tiền hiện tại.
- **Thành phần liên quan:**
 - Khách hàng (User): người thực hiện nhấn tin yêu cầu xem menu.
 - Giao diện người dùng (Telegram UI): khung chat nơi người dùng tương tác.

- Bot Controller: bộ điều phối xử lý ngôn ngữ tự nhiên (NLP) thông qua OpenAI.
- Menu Service: dịch vụ xử lý logic tra cứu món ăn.
- MongoDB: hệ thống cơ sở dữ liệu lưu trữ thông tin thực đơn.

• **Luồng tương tác:**

1. Khách hàng nhắn tin yêu cầu xem menu (VD: "Cho xem thực đơn").
2. Telegram UI gửi yêu cầu (Text) lên Bot Controller.
3. Bot Controller dùng AI phân tích ý định và kích hoạt công cụ `searchMenu`.
4. Menu Service truy vấn MongoDB lấy danh sách món ăn đang khả dụng.
5. MongoDB trả dữ liệu thô về cho Menu Service.
6. Menu Service định dạng dữ liệu thành chuỗi văn bản và trả về cho Bot Controller.
7. Bot tổng hợp câu trả lời thân thiện và gửi lại cho khách hàng qua Telegram UI.

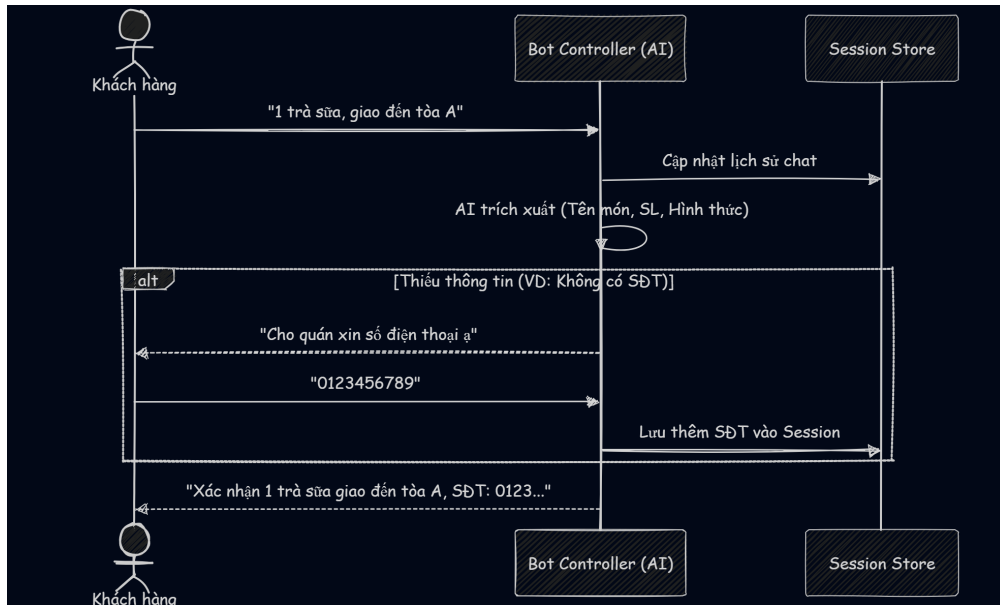
• **Activity Diagram:**



Hình 4: Activity Diagram - Tra cứu thực đơn

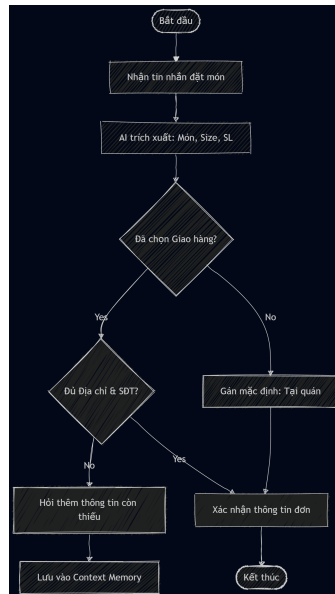
5.2 Đặt món và Cung cấp thông tin giao hàng

• **Sequence Diagram:**



Hình 5: Sequence Diagram - Đặt món và Cung cấp thông tin

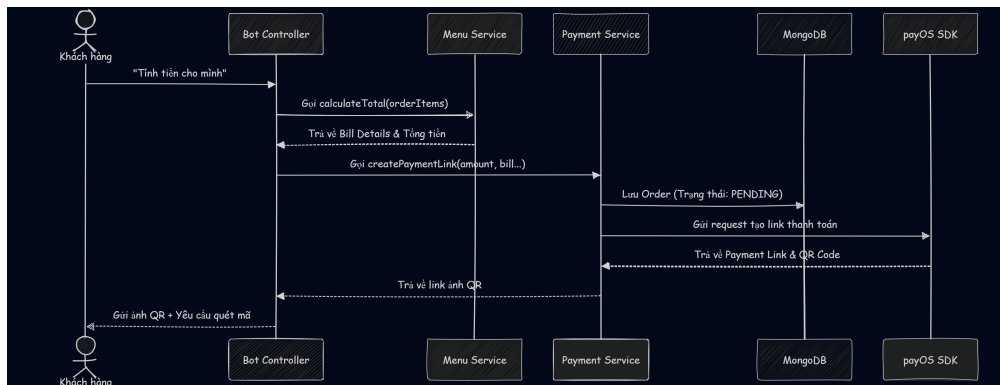
- **Mục đích:** Thể hiện quy trình AI tiếp nhận yêu cầu đặt món, trích xuất thực thể (Tên món, Số lượng, Size) và thu thập thông tin giao hàng nếu khách chọn mang đi.
- **Thành phần liên quan:**
 - Khách hàng (User): người thực hiện nhấn tin chốt món và cung cấp địa chỉ.
 - Bot Controller (AI): hệ thống tự động nhận diện và hỏi đáp.
 - Session Store (Context Memory): bộ nhớ RAM tạm thời lưu trữ ngữ cảnh hội thoại.
- **Luồng tương tác:**
 1. Khách hàng nhấn tin báo món muốn đặt.
 2. Bot Controller nhận tin, cập nhật vào Session Store.
 3. Bot phân tích để trích xuất các thông tin cần thiết.
 4. Bot kiểm tra hình thức nhận hàng. Nếu khách chọn "Giao hàng tận nơi" nhưng thiếu địa chỉ/SĐT, Bot sẽ đặt câu hỏi yêu cầu bổ sung.
 5. Khách hàng phản hồi thông tin còn thiếu.
 6. Bot lưu trữ các thông tin này vào Session Store và gửi tin nhắn xác nhận lại toàn bộ thông tin với khách hàng.
- **Activity Diagram:**



Hình 6: Activity Diagram - Đặt món và Cung cấp thông tin

5.3 Yêu cầu tính tiền và Tạo mã QR thanh toán

- Sequence Diagram:



Hình 7: Sequence Diagram - Tạo mã QR thanh toán

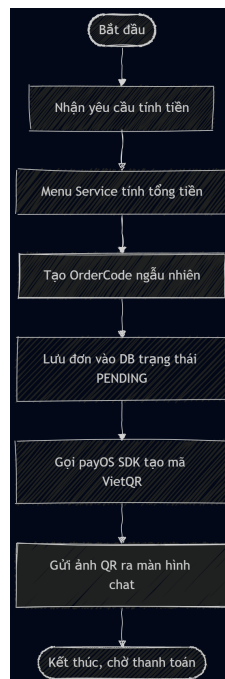
- **Mục đích:** Thể hiện quy trình tính toán tổng tiền dựa trên giỏ hàng, lưu thông tin vào cơ sở dữ liệu và gọi API payOS để sinh mã QR VietQR động.
- **Thành phần liên quan:**
 - Khách hàng (User): người yêu cầu chốt đơn.
 - Bot Controller: bộ điều phối chính.
 - Menu Service & Payment Service: dịch vụ xử lý tính tiền và tạo link thanh toán.

- payOS SDK: cổng thanh toán hỗ trợ sinh mã QR.
- MongoDB: nơi lưu trữ đơn hàng đang chờ thanh toán (PENDING).

- **Luồng tương tác:**

1. Khách hàng nhắn tin yêu cầu tính tiền.
2. Bot gọi Menu Service để tính tổng tiền và tạo hóa đơn chi tiết.
3. Bot gọi Payment Service để khởi tạo quy trình thanh toán.
4. Payment Service tạo mã OrderCode ngẫu nhiên và lưu đơn hàng vào MongoDB với trạng thái **PENDING**.
5. Payment Service gửi Request sang payOS SDK.
6. payOS trả về liên kết thanh toán và chuỗi dữ liệu mã QR.
7. Bot Controller nhận ảnh QR và gửi ra màn hình Telegram kèm hướng dẫn quét mã.

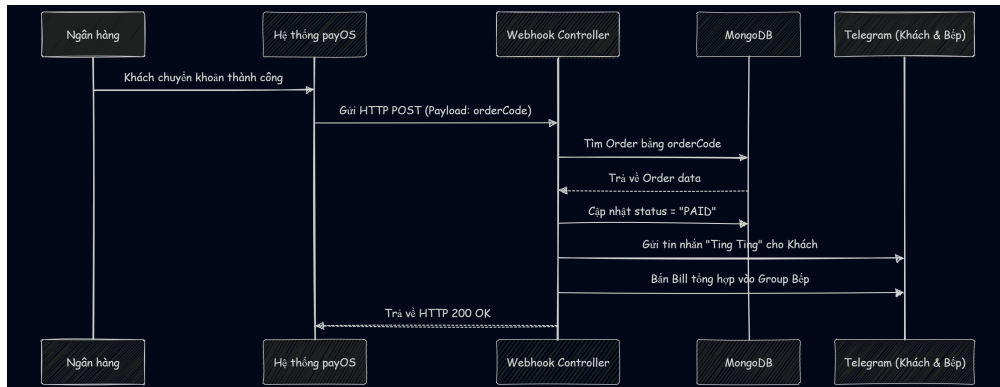
- **Activity Diagram:**



Hình 8: Activity Diagram - Tạo mã QR thanh toán

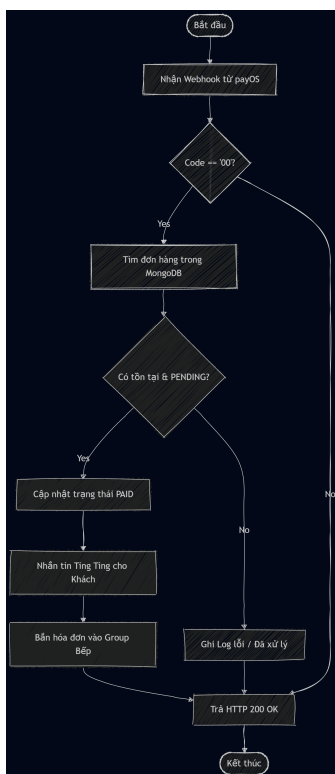
5.4 Xử lý Webhook và Thông báo Bếp (Chốt đơn tự động)

- **Sequence Diagram:**



Hình 9: Sequence Diagram - Xử lý Webhook chốt đơn

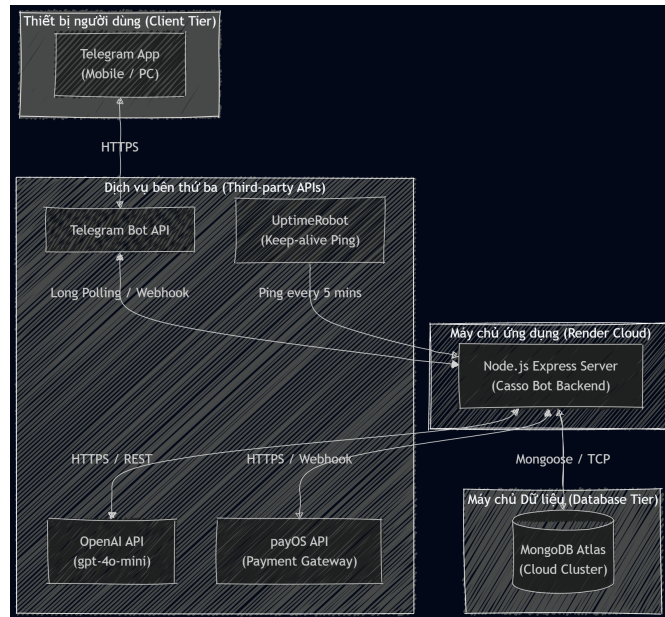
- **Mục đích:** Thể hiện quy trình tự động hoàn tất 100% khâu đối soát tài chính và chuyển giao thông tin làm món từ khách hàng đến thẳng nhà bếp mà không cần con người can thiệp.
- **Thành phần liên quan:**
 - Hệ thống ngân hàng & payOS: Đơn vị trung gian xử lý dòng tiền và kích hoạt Webhook.
 - Webhook Controller: API Endpoint trên server dùng để lắng nghe giao dịch.
 - MongoDB: cơ sở dữ liệu để tìm và cập nhật trạng thái đơn.
 - Giao diện Telegram (Khách & Bếp): nơi nhận thông báo cuối cùng.
- **Luồng tương tác:**
 1. Khách hàng quét mã QR chuyển khoản qua App ngân hàng thành công.
 2. payOS kích hoạt HTTP POST Request (Webhook) mang theo OrderCode gửi về Webhook Controller của Server.
 3. Controller truy xuất MongoDB tìm đơn hàng tương ứng với OrderCode.
 4. Nếu đơn hàng hợp lệ, Controller cập nhật trạng thái thành PAID vào DB.
 5. Controller gọi Telegram API gửi tin nhắn "Ting Ting" cảm ơn Khách hàng.
 6. Controller định dạng lại Bill tổng hợp (gồm món và địa chỉ) và bắn trực tiếp vào Group chat của bộ phận Bếp.
 7. Server phản hồi HTTP 200 OK để kết thúc kết nối với payOS.
- **Activity Diagram:**



Hình 10: Activity Diagram - Xử lý Webhook chốt đơn

6 Deployment View (Kiến trúc triển khai)

Kiến trúc triển khai (Deployment View) mô tả cách thức hệ thống phần mềm được cài đặt và vận hành trên các môi trường phần cứng hoặc nền tảng đám mây (Cloud Infrastructure) trong thực tế.



Hình 11: Deployment Diagram - Kiến trúc triển khai hệ thống

Mô tả các Node vật lý / Đám mây:

- **Client Tier (Thiết bị người dùng):** Giao diện giao tiếp duy nhất của người dùng là ứng dụng Telegram (trên Mobile hoặc Desktop). Khách hàng không kết nối trực tiếp với Server của hệ thống mà thông qua hạ tầng mạng của Telegram.
- **Third-party APIs (Dịch vụ bên thứ ba):**
 - **Telegram Bot API:** Đóng vai trò cầu nối, chuyển tiếp tin nhắn từ người dùng về Server và ngược lại.
 - **OpenAI API:** Máy chủ của OpenAI nhận các đoạn hội thoại (Prompt) từ Server để phân tích ngữ cảnh, xử lý NLP và trả về kết quả định tuyến.
 - **payOS API:** Cổng thanh toán xử lý yêu cầu tạo link VietQR và bắn Webhook ngược lại Server khi có biến động số dư.
 - **UptimeRobot (Monitoring/Ping Service):** Dịch vụ bên thứ ba hoạt động độc lập, tự động gửi HTTP Request (Keep-alive ping) định kỳ mỗi 5 phút đến Server ứng dụng để đánh thức hệ thống.
- **Cloud Server (Máy chủ ứng dụng):** Toàn bộ mã nguồn Node.js (Backend) được đóng gói và triển khai (Deploy) trên nền tảng Cloud **Render**. *Giải pháp tối ưu hóa (DevOps):* Do sử dụng gói dịch vụ miễn phí (Free Tier) có cơ chế tự động ngừng động (Spin down) sau

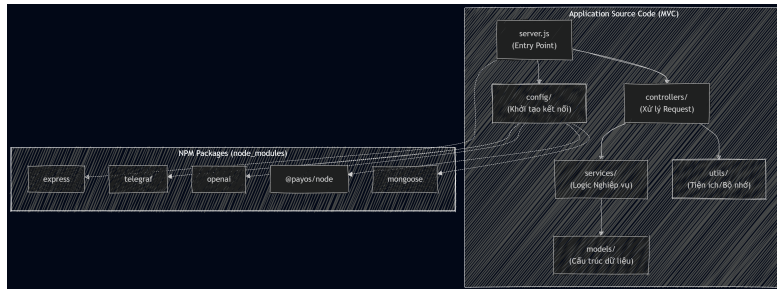


15 phút không hoạt động, hệ thống đã tích hợp **UptimeRobot** để liên tục "ping" vào Server. Điều này giúp ép Server luôn trong trạng thái thức (Awake), đảm bảo Bot luôn phản hồi tin nhắn của khách hàng ngay lập tức (Real-time) 24/7 mà không bị độ trễ khởi động (Cold Start - thường mất từ 30 đến 50 giây).

- **Database Tier (Máy chủ Dữ liệu):** Sử dụng cụm máy chủ đám mây **MongoDB Atlas** để lưu trữ thực đơn (Menu) và các hóa đơn chờ thanh toán (Pending Orders), đảm bảo dữ liệu không bao giờ bị mất ngay cả khi Server ứng dụng khởi động lại hoặc gặp sự cố.

7 Development View (Kiến trúc phát triển)

Kiến trúc phát triển (Development View) mô tả hệ thống dưới góc nhìn của lập trình viên, thể hiện cách mã nguồn được tổ chức thành các module, package và sự phụ thuộc vào các thư viện bên ngoài.



Hình 12: Development / Component Diagram - Tổ chức mã nguồn

Mô tả tổ chức mã nguồn và Thư viện phụ thuộc:

- **Kiến trúc thư mục nội bộ:** Hệ thống tuân thủ chặt chẽ mô hình **MVC** (Model-View-Controller) kết hợp **Service Pattern**. Mã nguồn được chia thành các gói độc lập: **config/** (khởi tạo API key), **controllers/** (điều hướng luồng tin nhắn và webhook), **services/** (chứa logic tính toán và gọi hàm) và **models/** (định nghĩa cấu trúc cơ sở dữ liệu).
- **Quản lý Dependency (NPM Packages):** Dự án phụ thuộc vào các thư viện mã nguồn mở quan trọng được quản lý qua **package.json**:
 - **express:** Quản lý HTTP Server và thiết lập Endpoint cho Webhook.
 - **telegraf:** Framework tối ưu cho việc tương tác với Telegram Bot API.
 - **openai:** SDK chính thức để gọi các mô hình LLM (gpt-4o-mini).
 - **@payos/node:** SDK cung cấp các phương thức bảo mật để tạo QR và xác thực chữ ký Webhook.
 - **mongoose:** Trình mô hình hóa đối tượng (ODM) giúp giao tiếp với MongoDB thông qua các Schema định nghĩa sẵn.



8 Kết luận và Hướng phát triển

8.1 Kết quả đạt được

- Tích hợp thành công Mô hình LLM gpt-4o-mini với độ chính xác cao trong việc nhận diện Entity (Món ăn, Size, Số lượng) từ ngôn ngữ tự nhiên tiếng Việt.
- Xây dựng thành công hệ thống Guardrails chặt chẽ thông qua System Prompt, bảo vệ Bot khỏi các luồng hội thoại độc hại.
- Xử lý trơn vẹn luồng thanh toán Tự động thông qua payOS, khắc phục hoàn toàn rủi ro thất thoát dữ liệu bằng cách lưu Order xuống MongoDB thay vì lưu trữ tạm trên RAM.

8.2 Hạn chế và Hướng phát triển

- **Hạn chế Context Window:** Việc lưu toàn bộ lịch sử chat vào Session có thể gây tốn kém chi phí Token API nếu cuộc hội thoại quá dài. Giải pháp: Cần thiết lập cơ chế tự động cắt tỉa (Trim) tin nhắn cũ.
- **Quản lý Menu quy mô lớn:** Hiện tại việc tìm kiếm món sử dụng Regex. Nếu Menu mở rộng lên hàng nghìn món, cần tích hợp cơ sở dữ liệu Vector (Vector Database) và kỹ thuật RAG (Retrieval-Augmented Generation) để tối ưu hóa việc tìm kiếm ngữ nghĩa.
- **Dashboard Quản trị:** Xây dựng thêm một Web Application (Frontend bằng Next.js hoặc React) để chủ quán có thể tự thêm/sửa món trên Menu mà không cần can thiệp trực tiếp vào Database.